

AGEX Development Harness

Codex / Claude / AI Coding Agent 통합 개발 지침 체계

1. 목적

AGEX는 문서량이 방대하기 때문에 모든 Coding Agent에게 30종 문서를 매번 전부 읽히는 방식은 비효율적이다.

따라서 AGEX 개발은 다음 구조를 사용한다.

AGENTS.md

↓

MASTER.md

↓

INDEX.md

↓

Relevant Domain Specification

↓

Canonical Schema

↓

Implementation

즉 Coding Agent는 항상:

1. 최상위 규칙을 먼저 읽고
2. 현재 작업 Domain을 식별하고
3. 필요한 문서만 선택적으로 읽고
4. Canonical Schema를 확인하고
5. 실제 코드를 수정한다.

이 방식을 AGEX의 공식 **AI Coding Harness**로 정의한다.

2. 권장 프로젝트 구조

agex/

|

|— AGENTS.md

|— CLAUDE.md

|— MASTER.md

|

|— docs/

| |— INDEX.md

| |— GLOSSARY.md

| |

| |— constitution/

| | |— 01-product-vision.md

| | |— 02-product-philosophy.md

| |

| |— architecture/

| | |— 03-product-architecture.md

| | |— 04-core-architecture.md

| | |— 05-runtime-architecture.md

| | |— 14-security-architecture.md

| | |— 15-multi-tenant-architecture.md

| | |— 16-deployment-architecture.md

| | |— 17-governance.md

| |

- | └─ domains/
 - | | └─ 06-agent-framework.md
 - | | └─ 07-workflow-engine.md
 - | | └─ 08-knowledge-engine.md
 - | | └─ 09-memory-engine.md
 - | | └─ 10-plugin-framework.md
 - | | └─ 11-marketplace.md
 - | | └─ 12-sdk.md
 - | | └─ 13-api-specification.md
 - | |
 - | └─ supplemental/
 - | | └─ S-03-canonical-resource-schema.md
 - | | └─ S-04-model-gateway.md
 - | | └─ S-05-iam.md
 - | | └─ S-06-billing-entitlement.md
 - | |
 - | └─ planning/
 - | | └─ 18-product-roadmap.md
 - | | └─ 19-development-roadmap.md
 - | | └─ 20-wbs.md
 - | | └─ 21-development-plan.md
 - | | └─ 22-resource-plan.md
 - | | └─ 23-cost-estimation.md
 - | |
 - | └─ business/

```
| | └─ 24-business-model.md
| |
| └─ guides/
|     └─ 25-technical-white-paper.md
|     └─ 26-product-documentation.md
|     └─ 27-operations-guide.md
|     └─ 28-api-guide.md
|     └─ 29-administrator-guide.md
|     └─ 30-developer-guide.md
|
└─ specs/
    └─ schemas/
    └─ permissions/
    └─ events/
    └─ fixtures/
    |
    └─ adr/
    └─ src/
    └─ tests/
    └─ tools/
```

3. AGENTS.md

Codex 및 공통 Coding Agent 최상위 지침

아래 내용을 프로젝트 루트의 AGENTS.md로 사용한다.

```
# AGEX Development Instructions
```

- lifecycle state
- permission
- event
- model capability
- workflow step type
- plugin side-effect class
- error category

search the canonical specifications first.

If the concept is missing, explicitly identify the specification gap rather than inventing a local convention silently.

19. AI coding-agent behavior

When implementing a task:

1. State which governing specification applies.
2. Inspect relevant existing code.
3. Identify any mismatch with the specification.
4. Prefer the specification over accidental legacy implementation.
5. Make the smallest coherent change that preserves architectural boundaries.
6. Add tests for the contract being implemented.

7. Report any unresolved architectural conflict.

Do not perform unrelated refactors unless they are required to satisfy the specification.

4. CLAUDE.md

Claude Code용 프로젝트 지침

Claude Code에는 별도 파일을 두되 AGENTS.md와 규칙을 중복 유지하지 않는 것이 중요하다.

권장:

```
# AGEX Claude Code Instructions
```

Before doing any implementation work, read:

```
@AGENTS.md
```

The instructions in AGENTS.md are the primary development rules for this repository.

Also follow the rules below.

```
## Claude-specific working rules
```

1. Read `/MASTER.md` before architectural or cross-domain changes.
2. Use `/docs/INDEX.md` to locate only the specifications relevant to the current task.

3. Do not load all AGEX documents unless the task spans the entire platform.
4. Do not infer missing platform contracts from existing code alone.
5. If implementation and specification conflict, identify the mismatch before preserving legacy behavior.
6. Do not introduce a new Resource, Permission, Event, lifecycle state, or public API shape without checking the canonical specification.
7. When making a public contract change, include schema and test changes in the same task.
8. Never bypass Runtime, Model Gateway, IAM, Tenant Isolation, or Security boundaries for implementation convenience.
9. Do not replace explicit AGEX domain concepts with generic maps or `any` objects simply to move faster.
10. Prefer a narrow, specification-compliant change over a broad speculative redesign.

Required response before major implementation

For substantial work, briefly identify:

- governing AGEX specification
- affected domain(s)
- affected schemas/APIs/events
- security/tenant implications
- tests required

Then implement.

Specification gap behavior

If a necessary contract is genuinely undefined:

- do not silently invent one;
- identify the missing contract;
- propose the smallest specification-compatible option;
- isolate any provisional implementation behind a clearly named interface or TODO.

Completion check

Before declaring a task complete, verify:

- specification compliance
- tenant isolation
- permission/policy handling
- versioning implications
- error handling
- observability
- tests

